VAULTSPHERE

A Secure Multi-Tenant SaaS Platform with CI/CD and Auto-Scaling

By GROUP 34:

- Vats Pratap Singh (22BSA10234)
- Keshav Shree (22BSA10129)
- Navika Mehrotra (22BSA10199)
- Sourabh Kumrawat (22BSA10112)
- Disha Jadhav (22BSA10061)

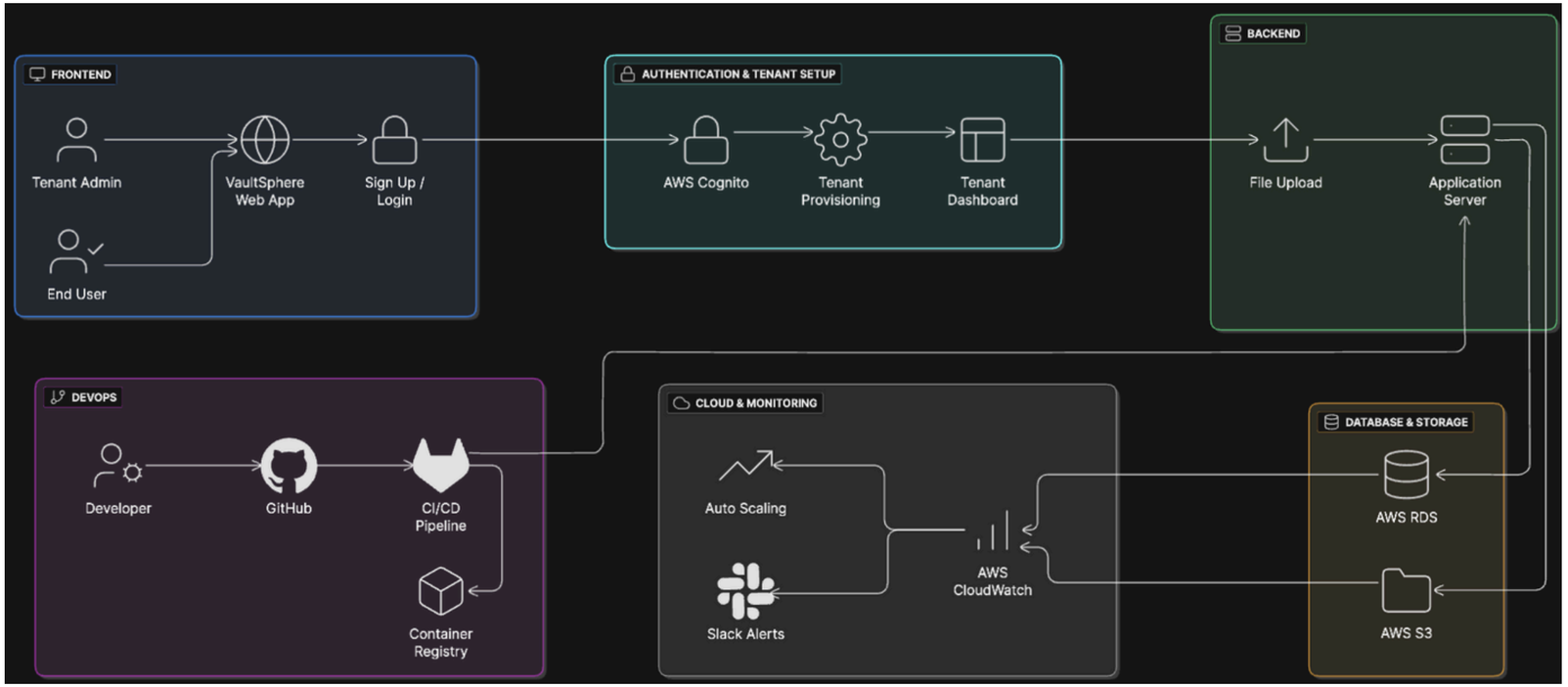
PROBLEM STATEMENT

Companies deploying SaaS solutions often face challenges due to the need to manage separate deployments for each client, leading to increased maintenance costs and higher operational risks. Manual deployment workflows add to the inefficiency, being slow, error-prone, and difficult to scale. Inadequate system designs lacking proper data isolation and access control can expose sensitive information, resulting in serious security breaches. Furthermore, such applications often struggle to dynamically scale during peak usage, causing downtime and performance bottlenecks. Additionally, the absence of centralized monitoring and debugging tools in distributed environments makes it difficult to identify and resolve issues effectively.

OBJECTIVE

1. Build a cloud-native SaaS platform capable of serving multiple organizations (tenants) from a single codebase.
2. Ensure data security and isolation for each tenant using proper multi-tenant design.
3. Implement CI/CD pipelines for automated testing and seamless deployment of new features.
4. Enable real-time monitoring and auto-scaling of application resources to support varying loads efficiently.
5. Use AWS services and Infrastructure as Code (Terraform) to deploy and manage scalable infrastructure.

WORKFLOW


FRONTEND
Tenant Admin
End User
VaultSphere Web App
Sign Up / Login
AUTHENTICATION & TENANT SETUP
AWS Cognito
Tenant Provisioning
Tenant Dashboard
BACKEND
File Upload
Application Server
DEVOPS
Developer
GitHub
CI/CD Pipeline
Container Registry
CLOUD & MONITORING
Auto Scaling
Slack Alerts
AWS CloudWatch
DATABASE & STORAGE
AWS RDS
AWS S3

PROPOSED WORK & METHODOLOGY

1. Frontend: Multi-tenant UI with secure login, dashboard per tenant using React.js/Next.js
2. Backend: RESTful APIs with JWT / Cognito auth, tenant-aware routing.
3. Database: Tenant isolation using schema or row-level separation using AWS RDS/ PostgreSQL/ MySQL.
4. CI/CD: GitHub Actions or Jenkins for automated testing and deployments.
5. Cloud Infra: Container orchestration, domain management, IaC setup using AWS ECS/EKS, S3, RDS, AWS Cognito, Route 53, setup using Terraform respectively.
6. Monitoring: Real-time tracking using Prometheus & Grafana
7. Auto-Scaling: Based on CPU/Memory/Request metrics using CloudWatch Metrics

NOVELTY

- Supports secure multi-tenancy out of the box
- Infrastructure defined as code using Terraform
- Full-stack CI/CD integration for both frontend and backend
- Real-time performance monitoring with scaling on demand
- Modular structure for adding tenant-specific custom features

REAL-TIME USAGE

- Multi-tenant platforms like CRM, SaaS tools, e-commerce backends
- Use cases:
  - B2B SaaS products serving multiple companies
  - Educational platforms with different schools as tenants
  - Healthcare systems with isolated hospital databases
- Scalable for any industry with tenant-based access control

THANK YOU!